

Understanding ChromaDB

ChromaDB is an open-source **vector database** built specifically for AI applications. It lets you store text (or any data) alongside its embedding vector, then search by *meaning* rather than keywords — all with a simple Python API.

Think of ChromaDB as a filing cabinet where each folder knows what it means. You can ask "find me everything about financial risk" and it returns the closest matches even if those exact words never appear.

1. Key Concepts

Concept	What it means	Analogy
Client	The entry point — connects you to ChromaDB (local or remote).	Opening the filing cabinet
Collection	A named group of related documents. Like a table in SQL.	One drawer in the cabinet
Document	A piece of text you want to store and search.	A single file in the drawer
Embedding	A list of numbers (vector) that captures the meaning of a document. ChromaDB can generate these for you automatically.	The label on the file
ID	A unique string identifier for each document.	The file number
Metadata	Extra key-value data attached to each document (e.g. source, category, date).	Sticky notes on the file
Distance	How far apart two embeddings are. Lower = more similar.	How far two files are from each other

How ChromaDB fits into an AI project

ChromaDB sits between your raw text and your AI model. You feed it text → it stores both the text and its vector. At query time, your search phrase is converted to a vector and ChromaDB finds the nearest stored vectors — returning the most semantically relevant documents in milliseconds, even across thousands of records.

Your Text	→	Embedding Model	→	ChromaDB stores vector + text + metadata
Your Query	→	Embedding Model	→	ChromaDB returns top-K nearest docs

2. Installation & Setup

ChromaDB runs entirely in Python with no external server required for local use. A single pip command is all you need.

Install

terminal

```
pip install chromadb
```

Two client modes

Mode	Code	When to use
In-memory (ephemeral)	<pre>import chromadb client = chromadb.Client()</pre>	Quick experiments, tests. Data lost when script ends.
Local persistent (recommended)	<pre>client = chromadb.PersistentClient(path="./chroma_store")</pre>	Development & learning. Data saved to disk automatically.
HTTP client (production)	<pre>client = chromadb.HttpClient(host="localhost", port=8000)</pre>	Production. Run ChromaDB as a separate server process.

Creating a local persistent client

```
setup.py

import chromadb

# Data is saved in ./chroma_store on your disk
client = chromadb.PersistentClient(path="./chroma_store")

# Verify it's working
print(client.heartbeat()) # returns a timestamp
```

Tip: The **path** folder is created automatically if it doesn't exist. Every collection and document you add is persisted there immediately — no manual save step needed.

3. Collections & Adding Documents

A **collection** is the core container in ChromaDB — equivalent to a table in a relational database. Each document you add needs three things: a unique **ID**, the **text** itself, and optional **metadata** (key-value pairs for filtering later).

Create a collection

```
collections.py

# get_or_create avoids errors if the collection already exists
collection = client.get_or_create_collection(
    name="my_documents",
    metadata={"hnsw:space": "cosine"} # use cosine distance (recommended)
)
```

Add 20 documents with metadata

Each document gets a unique ID, a text string, and a metadata dict. ChromaDB auto-generates the embedding using its default model (**all-MiniLM-L6-v2**) unless you supply your own vectors.

Step 1 — define the 20 documents (5 per topic)

add_documents.py — part 1/3

```
# finance (docs 1-5), space (6-10), health (11-15), technology (16-20)
documents = [
    "Inflation rates rose sharply in Q3 due to energy prices.",
    "The Federal Reserve raised interest rates by 75 basis points.",
    "Stock markets fell amid recession fears and dollar strength.",
    "Gold is traditionally seen as a hedge against inflation.",
    "Venture capital funding in AI startups doubled this year.",
    "NASA launched a new Mars rover to search for microbial life.",
    "The James Webb telescope revealed galaxies from 13 billion years ago.",
    "SpaceX successfully landed its reusable Starship booster.",
    "Astronomers detected gravitational waves from merging black holes.",
    "A new exoplanet in the habitable zone was found 40 light-years away.",
    "Researchers developed a vaccine using mRNA technology for cancer.",
    "A clinical trial showed 90% efficacy for the new diabetes drug.",
    "CRISPR gene editing corrected a hereditary blindness condition.",
    "A study linked gut microbiome diversity to reduced anxiety levels.",
    "AI models are now detecting tumours earlier than human radiologists.",
    "The new Python 3.12 release brings faster startup and better errors.",
    "Rust is gaining popularity for systems programming due to safety.",
    "Kubernetes 1.28 improved node resource management significantly.",
    "WebAssembly is enabling high-performance apps in the browser.",
    "Open-source LLMs like Llama 3 are closing the gap with GPT-4.",
]
```

Step 2 — define metadata for each document

add_documents.py — part 2/3

```
metadatas = [
    {"category": "finance", "source": "Reuters", "date": "2024-09-10"},
    {"category": "finance", "source": "Bloomberg", "date": "2024-09-15"},
    {"category": "finance", "source": "CNBC", "date": "2024-09-20"},
    {"category": "finance", "source": "FT", "date": "2024-09-22"},
    {"category": "finance", "source": "TechCrunch", "date": "2024-09-25"},
    {"category": "space", "source": "NASA", "date": "2024-08-01"},
    {"category": "space", "source": "ESA", "date": "2024-08-05"},
    {"category": "space", "source": "SpaceX", "date": "2024-08-10"},
    {"category": "space", "source": "Nature", "date": "2024-08-15"},
    {"category": "space", "source": "NASA", "date": "2024-08-20"},
    {"category": "health", "source": "NIH", "date": "2024-07-01"},
    {"category": "health", "source": "Lancet", "date": "2024-07-10"},
    {"category": "health", "source": "NEJM", "date": "2024-07-15"},
    {"category": "health", "source": "Nature", "date": "2024-07-20"},
    {"category": "health", "source": "Science", "date": "2024-07-25"},
    {"category": "technology", "source": "GitHub", "date": "2024-06-01"},
    {"category": "technology", "source": "Mozilla", "date": "2024-06-10"},
    {"category": "technology", "source": "CNCF", "date": "2024-06-15"},
    {"category": "technology", "source": "W3C", "date": "2024-06-20"},
    {"category": "technology", "source": "Meta AI", "date": "2024-06-25"},
]
```

Step 3 — add everything to the collection

add_documents.py — part 3/3

```
ids = [f"doc_{i}" for i in range(1, 21)] # doc_1 ... doc_20

collection.add(
    documents=documents,
    metadatas=metadatas,
    ids=ids
)

print(collection.count()) # → 20
```

*Note: ChromaDB auto-embeds the documents on **add()** using its built-in embedding function. No separate encode step needed.*

4. Running Semantic Queries

The **collection.query()** method takes a plain-text query, embeds it automatically, and returns the most semantically similar documents. You choose how many results to return with **n_results**.

Query anatomy

Parameter	Type	What it does
query_texts	list[str]	One or more query strings. Each gets embedded and searched.
n_results	int	How many results to return per query.
where	dict	Metadata filter (covered in Section 5).
include	list[str]	What to include in results: 'documents', 'distances', 'metadatas'.

Three example queries

queries.py

```
# Query 1: Finance topic
results_1 = collection.query(
    query_texts=["impact of interest rates on stock market"],
    n_results=3,
    include=["documents", "distances", "metadatas"]
)

# Query 2: Space topic
results_2 = collection.query(
    query_texts=["discoveries about distant galaxies and planets"],
    n_results=3,
    include=["documents", "distances", "metadatas"]
)

# Query 3: Health/AI crossover
results_3 = collection.query(
    query_texts=["AI applications in medical diagnosis"],
    n_results=3,
    include=["documents", "distances", "metadatas"]
)
```

Inspecting the results

inspect_results.py

```
def show_results(results, title):
    print(f"\n=== {title} ===")
    docs = results['documents'][0]
    distances = results['distances'][0]
    metas = results['metadatas'][0]
    for i, (doc, dist, meta) in enumerate(zip(docs, distances, metas), 1):
        print(f" #{i} dist={dist:.4f} [{meta['category']}] {doc[:70]}...")

show_results(results_1, 'Interest rates & stocks')
show_results(results_2, 'Galaxies & planets')
show_results(results_3, 'AI in medical diagnosis')
```

Expected output (approximate)

output

```
=== Interest rates & stocks ===
#1 dist=0.0821 [finance] The Federal Reserve raised interest rates...
#2 dist=0.1203 [finance] Stock markets fell amid recession fears...
#3 dist=0.2154 [finance] Inflation rates rose sharply in Q3...

=== Galaxies & planets ===
#1 dist=0.0912 [space] The James Webb telescope revealed galaxies...
#2 dist=0.1044 [space] A new exoplanet in the habitable zone...
#3 dist=0.1788 [space] Astronomers detected gravitational waves...

=== AI in medical diagnosis ===
#1 dist=0.0756 [health] AI models are now detecting tumours...
#2 dist=0.1893 [health] Researchers developed a vaccine using mRNA...
#3 dist=0.2341 [technology] Open-source LLMs like Llama 3...
```

Reading distances: ChromaDB returns cosine distances (0 = identical, 2 = opposite). A distance below ~0.15 is a strong match. Above ~0.40 the semantic connection is weak. Notice the AI/medical query correctly pulls a health document AND a technology document — because both are genuinely related to the concept.

5. Metadata Filtering with Semantic Search

ChromaDB lets you combine semantic search with **exact metadata filters** using the **where** parameter. This is powerful: you narrow the search space to a subset of documents before semantic ranking happens.

Filter syntax

Operator	Example	Meaning
\$eq	<code>{"category": {"\$eq": "finance"}}</code>	Exact match
\$ne	<code>{"category": {"\$ne": "space"}}</code>	Not equal
\$in	<code>{"source": {"\$in": ["Reuters", "FT"]}}</code>	Value in list
\$and	<code>{"\$and": [{...}, {...}]}</code>	Both conditions must be true
\$or	<code>{"\$or": [{...}, {...}]}</code>	Either condition must be true
Shorthand	<code>{"category": "finance"}</code>	Shortcut for \$eq — same result

Example 1 — Only search within finance documents

filter_finance.py

```
results = collection.query(
    query_texts=["impact of interest rates on the economy"],
    n_results=3,
    where={"category": "finance"}, # shorthand for $eq
    include=["documents", "distances", "metadatas"]
)

# Only the 5 finance docs are searched – space/health/tech docs are excluded
for doc, dist, meta in zip(
    results['documents'][0],
    results['distances'][0],
    results['metadatas'][0]
):
    print(f"[{meta['category']}] [{meta['source']}] dist={dist:.4f}")
    print(f" {doc[:80]}")
```

Example 2 — Filter by source AND category

filter_combined.py

```
# Only return health documents from Nature journal
results = collection.query(
    query_texts=["new discoveries in medical treatment"],
    n_results=2,
    where={
        "$and": [
            {"category": {"$eq": "health"}},
            {"source": {"$eq": "Nature"}}
        ]
    },
    include=["documents", "distances", "metadatas"]
)
```

Example 3 — Filter by multiple categories (\$in)

filter_in.py

```
# Search across both finance and technology documents only
results = collection.query(
    query_texts=["AI and automation changing the economy"],
    n_results=4,
    where={"category": {"$in": ["finance", "technology"]}},
    include=["documents", "distances", "metadatas"]
)

for doc, dist, meta in zip(
    results['documents'][0],
    results['distances'][0],
    results['metadatas'][0]
):
    print(f"dist={dist:.4f} [{meta['category']}] {doc[:70]}")
```

Important: The **where** filter runs *before* semantic ranking. ChromaDB first narrows the candidate pool by metadata, then finds the nearest vectors within that pool. This makes it fast even on large collections — you're only doing vector math on the filtered subset.

6. Other Useful Operations

other_operations.py

```
# Count documents in a collection
collection.count() # → 20

# Fetch a specific document by ID
collection.get(ids=["doc_1"])

# Update a document
collection.update(
    ids=["doc_1"],
    documents=["Updated text here"],
    metadatas=[{"category": "finance", "source": "Reuters", "date": "2024-10-01"}]
)

# Delete a document
collection.delete(ids=["doc_1"])

# Delete the entire collection
client.delete_collection("my_documents")

# List all collections
client.list_collections()
```

Quick Reference Card

Task	One-liner
Create persistent client	<code>chromadb.PersistentClient(path="./store")</code>
Create / open collection	<code>client.get_or_create_collection("name")</code>
Add documents	<code>col.add(documents=[...], metadatas=[...], ids=[...])</code>
Count documents	<code>col.count()</code>
Semantic query	<code>col.query(query_texts=["..."], n_results=3)</code>
Filter by category	<code>where={"category": "finance"}</code>
Filter: AND conditions	<code>where={"\$and": [{"cat": "finance"}, {"src": "FT"}]}</code>
Filter: value in list	<code>where={"category": {"\$in": ["finance", "health"]}}</code>
Fetch by ID	<code>col.get(ids=["doc_1"])</code>
Delete document	<code>col.delete(ids=["doc_1"])</code>